SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

Static files

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Static files

- 웹 서버와 정적 파일
- Static files 기본 경로
- Static files 추가 경로

Media files

- 이미지 업로드
- 업로드 이미지 제공
- 업로드 이미지 수정

참고

- upload_to 활용
- AWS 인프라 이해하기
- BaseModelForm

학습 시작

"내가 만든 웹 페이지, 왜 이렇게 밋밋할까? 밋밋한 내 웹사이트에 생기를 불어넣고 싶다면?"

- 우리가 매일 방문하는 웹사이트들은 화려한 이미지, 로고, 그리고 아름다운 디자인으로 가득 차 있습니다.
- 또한, 사용자들은 자신의 프로필 사진을 직접 올리고 변경하기도 합니다.
- 지금까지 우리가 만든 Django 페이지는 글자만 가득했습니다.
- CSS도, 이미지도, 사용자가 올린 파일도 없었죠. 과연 이런 요소들은 어떻게 웹 페이지에 적용되는 걸까요?

1. 이번 시간 학습을 마치면, 우리는 직접 만든 웹 페이지에 로고 이미지를 넣고, CSS 파일을 적용해 디자인을 꾸미며, 사용자가 직접 이미지를 업로드하는 게시판을 만들 수 있게 될 것입니다.
2. 웹 개발의 핵심인 정적 파일(Static files)과 미디어 파일(Media files) 관리법을 배워, 여러분의 웹사이트를 한층 더 다채롭게 만들게 될 것입니다.

학습 목표

- ✓ 정적 파일(Static)과 미디어 파일(Media)의 차이점을 명확히 설명할 수 있다.
- ✓ Django에서 정적 파일의 역할과 처리 과정을 이해하고, CSS, JS, 이미지 파일을 프로젝트에 적용할 수 있다.
- ✓ STATICFILES_DIRS를 설정하여 여러 경로에 있는 정적 파일을 효율적으로 관리할 수 있다.
- ✓ Django에서 미디어 파일의 처리 과정을 이해하고, 사용자가 업로드하는 파일을 서버에 저장하고 관리할 수 있다.
- ✓ MEDIA_ROOT와 MEDIA_URL을 설정하여 업로드 된 파일을 사용자에게 보여줄 수 있다.
- ✓ ImageField와 ModelForm을 활용하여 이미지 업로드 기능을 구현할 수 있다.

Static files

Static files란?

Static files

정적 파일

서버 측에서 변경되지 않고
고정적으로 제공되는 파일

마치 식당에 비치된 '인쇄된 메뉴판'과 같습니다. 메뉴판은 손님이 바뀐다고 해서 내용이 변하지 않고, 항상 동일한 모습으로 제공됩니다.

- 대표적인 종류: CSS 파일, JavaScript 파일, 이미지 파일, 폰트 파일

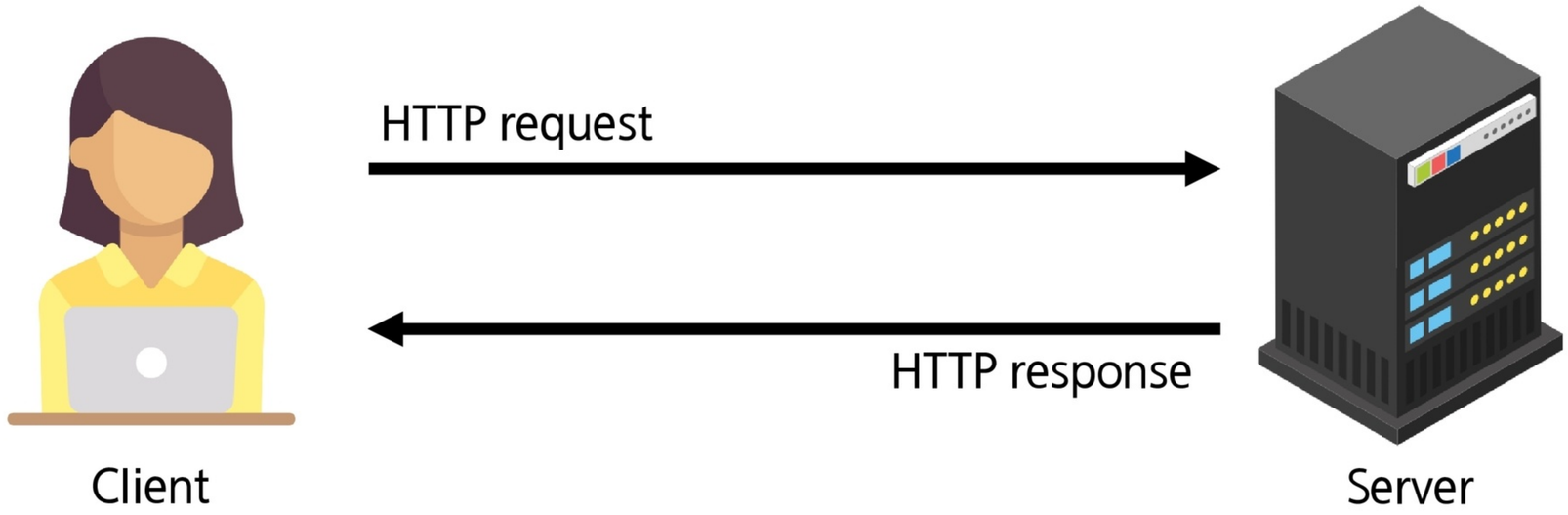
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

웹 서버와 정적 파일

웹 서버의 기본 역할

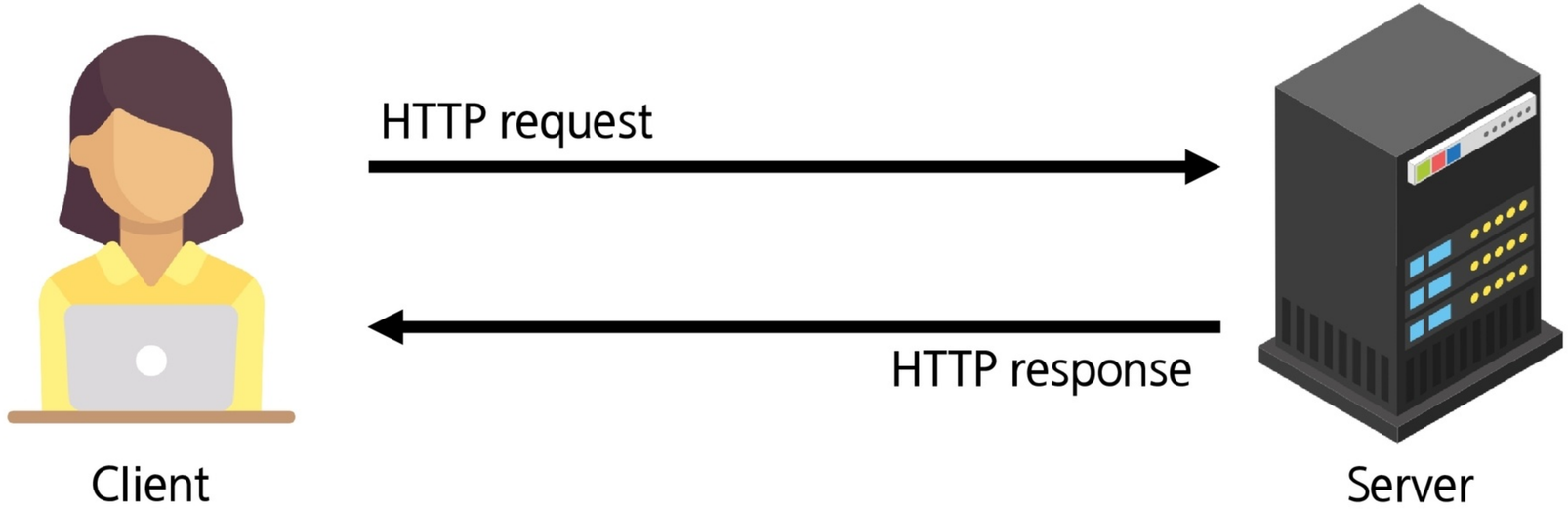
- "요청 받은 주소(URL)에 해당하는 자원을 찾아 응답해주는 것"
- 마치 도서관 사서에게 "A-3 구역의 3번째 책을 주세요"라고 요청하면, 사서가 해당 위치로 가서 책을 찾아 우리에게 건네 주는 것과 같음
  - 여기서 'A-3 구역의 3번째'가 바로 URL이고, '책'이 자원(Resource)에 해당



<그림1_Django_Static files_요청과 응답>

정적 파일과 URL의 관계

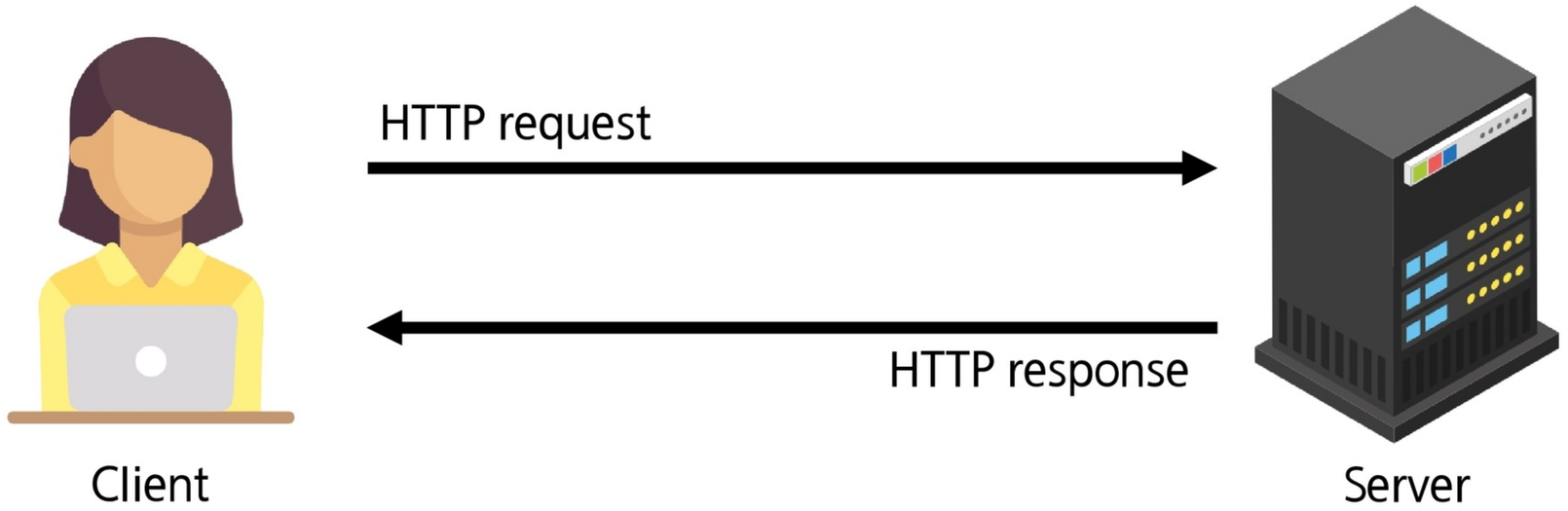
- 웹 서버가 제공하는 가장 기본적인 자원이 바로 정적 파일(Static Files)
- 정적 파일 제공
  - 웹 서버는 요청 받은 URL을 보고, 서버 컴퓨터의 특정 폴더에 저장된 CSS, JS, 이미지 같은 정적 파일을 찾아 제공
- URL의 필요성
  - 결국 "정적 파일이 사용자에게 보이려면, 그 파일에 접근할 수 있는 고유한 주소(URL)가 반드시 필요하다"는 의미



<그림1_Django_Static files_요청과 응답>

처리 과정 요약

1. **사용자**: 브라우저에 `http://example.com/images/logo.png` 라는 주소를 입력하며 이미지를 요청
2. **웹 서버**: `/images/logo.png` 라는 URL을 확인하고, 서버에 미리 약속된 폴더에서 `logo.png` 파일을 찾음
3. **웹 서버**: 찾은 이미지 파일을 HTTP 응답에 담아 사용자에게 전송
4. **사용자**: 브라우저가 응답 받은 이미지 파일을 화면에 보여줌



<그림1_Django_Static files_요청과 응답>

Static files 경로의 종류

1. 기본 경로
2. 추가 경로

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Static files 기본 경로

정적 파일이 위치해야 하는 기본 경로

Static files 기본 경로

app폴더/static/

기본 경로 CSS 스타일 제공하기 (1/3)

- articles/static/stylesheets/ 경로에 CSS 파일 배치하기

<그림2_Django_Static files_CSS 파일 배치>

기본 경로 CSS 스타일 제공하기 (2/3)

- 메인 페이지에서 CSS 파일 불러오기
- static files 경로는 DTL의 `static` tag를 사용해야 함
- built-in tag가 아니기 때문에 `load` tag를 사용해 `import` 후 사용 가능

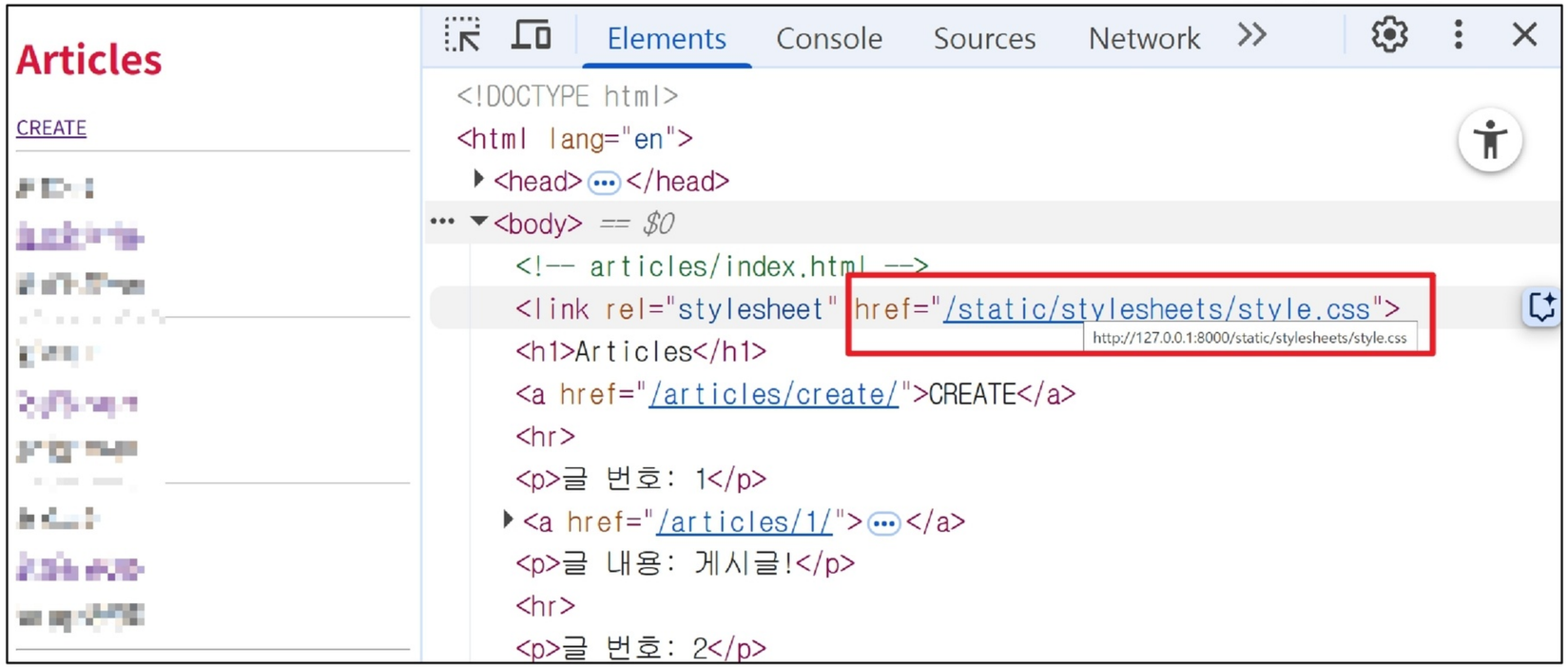
```
<!-- articles/index.html -->

{% load static %}

<link rel="stylesheet" href="{% static 'stylesheets/style.css' %}">
```

※ app폴더/static/ 경로 이후 경로를 작성합니다.

기본 경로 CSS 스타일 제공하기 (3/3)

- CSS가 적용 된 것을 확인
- static tag가 어떤 주소를 만들어 냈는지 확인

<그림3_Django_Static files_CSS URL 결과>



※ /static/으로 시작하는 웹 주소가 존재하는 것을 확인할 수 있습니다.

{% load %}

- 특정 라이브러리의 태그와 필터를 현재 템플릿에서 사용할 수 있도록 불러오는 역할
- `{% load static %}`은 `{% static %}` 태그를 사용하기 위해, Django 템플릿 시스템에 "이제부터 static 관련 태그를 사용하겠다."고 알려주는 선언문

※ load 태그는 템플릿 파일의 가장 상단에 한 번만 작성하면 됩니다.

※ `{% extends %}`를 사용하는 템플릿에서는 extends가 첫 줄이고 load는 그 아래에 씁니다. 또한 부모 템플릿에서 load 했더라도 자식 템플릿에서 다시 load 해야 합니다.

{% static '경로' %}

- settings.py 파일의 `STATIC_URL` 값을 앞에 붙여 정적 파일의 웹 경로(`/static/css/style.css`)를 만들어 줌
- 예를 들어, `STATIC_URL = 'static/'` 이고 CSS 파일이 static/css/style.css에 위치한다면, 경로가 필요한 위치에 `{% static 'css/style.css' %}`로 작성

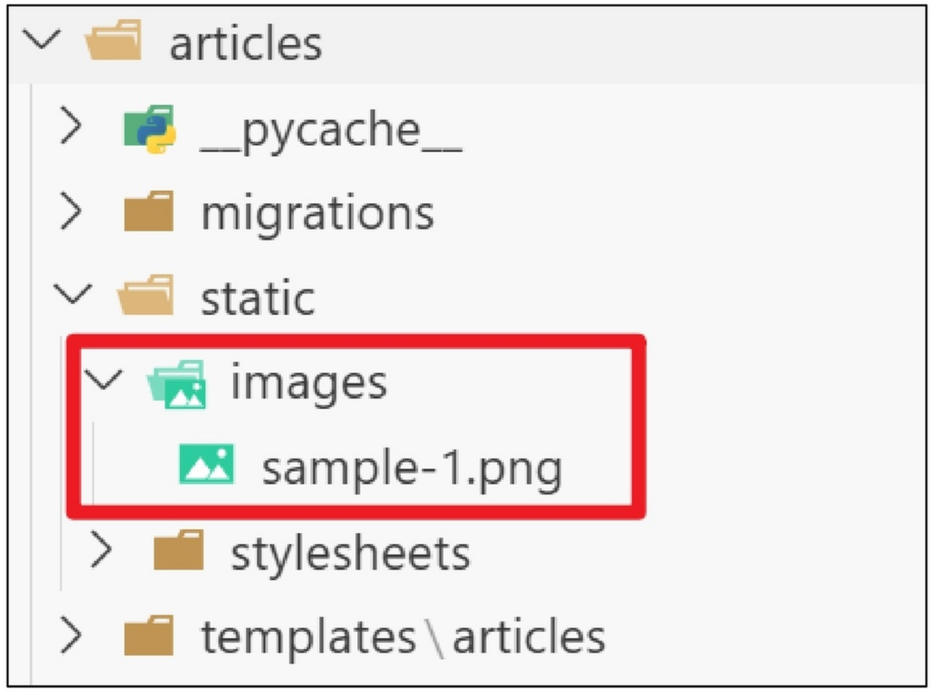
STATIC_URL이란?

STATIC_URL | 정적 파일의 '웹 주소'

- 웹 페이지에서 정적 파일에 접근할 때 사용할 URL의 시작 부분(접두사)을 지정하는 설정
- 이는 서버 컴퓨터에 저장된 실제 파일 경로가 아니라, 오직 웹(브라우저)에서만 사용되는 주소
- 마치 서버의 정적 파일 폴더에 `'static/'`이라는 웹 주소 별명을 붙여주는 것과 같습니다. 브라우저가 이 별명으로 파일을 요청하면, Django는 약속된 실제 폴더에서 파일을 찾아 응답해줍니다.
- 기본 값: `'static/'`

기본 경로 이미지 파일 제공하기 (1/3)

- `articles/static/images/` 경로에 이미지 파일 배치하기



<그림4_Django_Static files_이미지 파일 배치>

기본 경로 이미지 파일 제공하기 (2/3)

- DTL의 **static** tag를 사용해 이미지 경로 작성하기

```
<!-- articles/index.html -->


<img src="{% static 'images/sample-1.png' %}" alt="sample image">
```

기본 경로 이미지 파일 제공하기 (3/3)

- 제공된 이미지의 URL 확인
- http://127.0.0.1:8000/static/images/sample-1.png

<그림5_Django_Static files_이미지 파일 URL 결과>

정적파일 URL이 만들어지기까지

서버 주소 + STATIC_URL + 정적파일 경로

http://127.0.0.1:8000/static/images/sample-1.png

Static files 추가 경로

기본 경로 외 추가 경로 지정

Static files 추가 경로

`STATICFILES_DIRS`에 추가 경로 목록(리스트)을 설정

STATICFILES_DIRS란?

STATICFILES_DIRS | 기본 경로 외에 추가적으로 탐색할 경로의 목록을 지정하는 리스트

마치 기본 도서관 외에, 책을 보관하는 '외부 창고'들의 주소 목록을 알려주는 것과 같습니다.

Django는 이 목록에 있는 모든 폴더를 방문하여 필요한 정적 파일을 찾습니다.

이 설정은 여러 앱에서 공통으로 사용되는 CSS 프레임워크(예: Bootstrap)나 JavaScript 라이브러리처럼 특정 앱에 속하지 않는 정적 파일들을 프로젝트 최상위 폴더(BASE_DIR) 같은 곳에 모아두고 관리할 때 매우 유용합니다.

STATICFILES_DIRS 설정

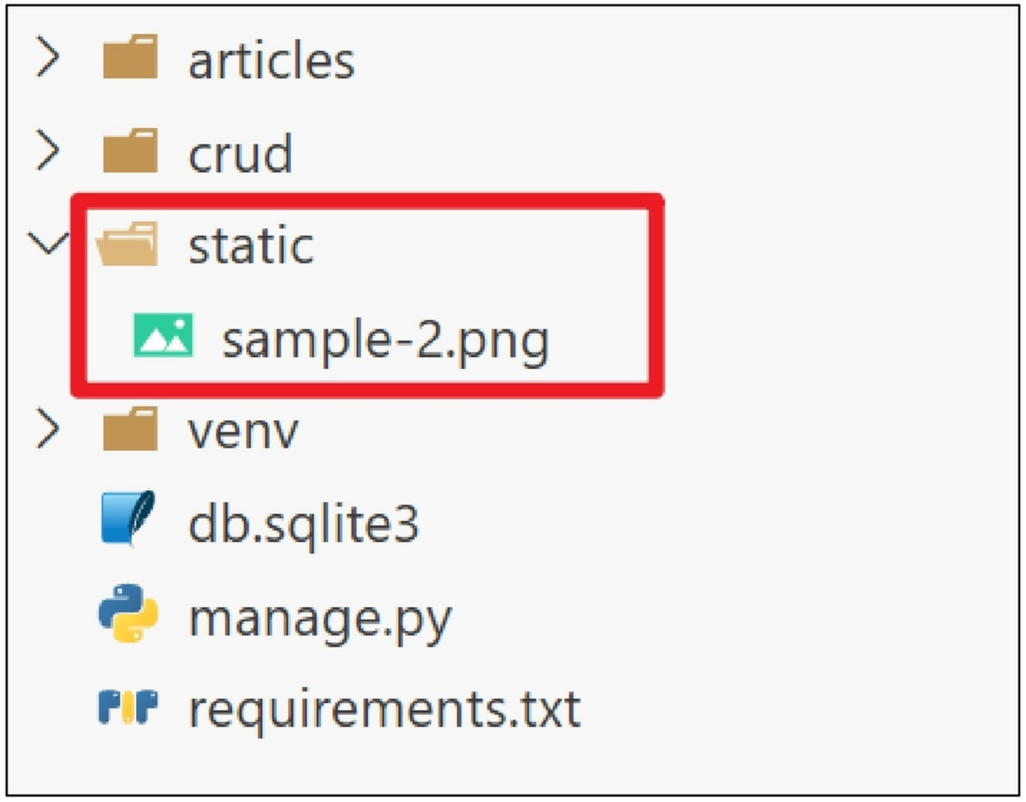
- 이처럼 설정하면, Django는 각 앱의 `static/` 폴더뿐만 아니라 프로젝트 최상위의 `static/` 폴더에서도 정적 파일을 찾게 됨

➢ Django는 `STATICFILES_DIRS`를 먼저, 그 다음 각 앱의 `static/` 폴더를 찾음

```
# settings.py

STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

추가 경로 이미지 파일 제공하기 (1/3)

- 설정한 추가 경로에 이미지 파일 배치하기



<그림6_Django_Static files_추가 이미지 파일 배치>

추가 경로 이미지 파일 제공하기 (2/3)

- DTL의 **static** tag를 사용해 이미지 경로 작성하기

```
<!-- articles/index.html -->

<img src="{% static 'sample-2.png' %}" alt="sample image">
```

추가 경로 이미지 파일 제공하기 (3/3)

- 제공된 이미지의 URL 확인
- http://127.0.0.1:8000/static/sample-2.png

<그림7_Django_Static files_추가 이미지 파일 URL 결과>

정적 파일의 핵심 원리: 주소(URL)가 있어야 찾아갈 수 있다.

- 컴퓨터에 파일이 존재하는 것만으로는 웹 페이지에 보일 수 없음
- 외부의 손님(브라우저)이 파일을 찾아올 수 있도록 반드시 '웹 주소(URL)'라는 문패를 달아줘야 함

구분	개념	비유
서버 (내 컴퓨터)	실제 파일 경로	"내 방 책상 위 세 번째 서랍"
웹 (인터넷 세상)	URL (웹 주소)	"서울시 강남구 테헤란로 999"

<표1_Django_Static files_정적 파일의 핵심 원리>

Media files

Media files란?

Media files

미디어 파일

사용자가 웹사이트를 통해
직접 업로드하는 파일

Static Files이 사이트 개발자가 미리 준비해 둔 '고정된' 파일이라면, Media Files는 사이트 운영 중 사용자에 의해 생성되고 변경되는 '동적인' 파일입니다.

마치 쇼핑몰의 기본 로고나 아이콘은 정적 파일이고, 사용자가 상품 후기를 작성하며 올리는 상품 사진은 미디어 파일인 것과 같습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이미지 업로드

ImageField()

- 이미지 파일을 업로드하기 위해 사용하는 Django 모델 필드
- 데이터베이스 저장 방식
  - 가장 중요한 특징은 이미지 파일 자체가 데이터베이스에 저장되는 것이 아니라는 점
  - 데이터베이스에는 `MEDIA_ROOT` 기준 상대 경로(예: images/sample.png)만 저장되고, 실제 파일은 서버의 특정 폴더(`MEDIA_ROOT`)에 저장

```python
# models.py

class Article(models.Model):
    # 이미지는 'MEDIA_ROOT경로/images/' 경로에 저장되고,
    # DB에는 'images/sample.png' 와 같은 경로 문자열이 저장됨
    image = models.ImageField(upload_to='images/')
```

※ upload_to는 선택 인자입니다.

미디어 파일을 제공하기 전 준비사항

1. settings.py에 `MEDIA_ROOT`, `MEDIA_URL` 설정
2. 작성한 MEDIA_ROOT와 MEDIA_URL에 대한 URL 지정

MEDIA_ROOT란?

MEDIA_ROOT | 미디어 파일의 '실제 창고' 주소

사용자가 업로드한 미디어 파일들이 서버 컴퓨터 **어디에 저장될지를 지정**하는 **절대 경로**입니다.

서버 내부에서만 사용하는 **물리적인 폴더 주소**로, Django는 파일을 저장하거나 읽어올 때 이 경로를 사용합니다.

```
# 프로젝트 폴더(BASE_DIR) 안에 'media'라는 폴더를 실제 저장소로 사용
MEDIA_ROOT = BASE_DIR / 'media'
```

MEDIA_URL이란?

MEDIA_URL | 미디어 파일의 '웹 주소' 별명

MEDIA_ROOT에 저장된 파일들을 웹 페이지에서 접근할 때 사용할 URL의 시작 부분을 지정합니다.

STATIC_URL과 동일한 역할로, 실제 저장 위치를 숨기고 웹에서 사용할 공개 주소 별명을 만들어주는 것입니다.

```
# media/ 라는 주소로 파일을 요청하면 MEDIA_ROOT 폴더에서 찾아 응답
MEDIA_URL = 'media/'
```

MEDIA_ROOT와 MEDIA_URL 설정 (1/2)

- MEDIA_ROOT는 파일을 저장하고 관리하기 위한 서버의 실제 경로이고,
  MEDIA_URL은 그 파일을 웹에서 보여주기 위한 가상의 주소

```
# settings.py

MEDIA_ROOT = BASE_DIR / 'media'

MEDIA_URL = 'media/'
```

MEDIA_ROOT와 MEDIA_URL 설정 (2/2)

- `settings.MEDIA_URL`: "media/로 시작하는 URL 요청이 오면,"
- `document_root=settings.MEDIA_ROOT`: "MEDIA_ROOT에 지정된 실제 폴더에서 파일을 찾아라."

```python
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

TIP

- Django 개발 서버는 정적 파일(static)은 자동으로 제공하지만, 사용자가 올린 미디어 파일(media)은 자동으로 제공하지 않습니다.
- 그래서 "media/로 요청이 오면 MEDIA_ROOT 폴더에서 찾아라"라는 규칙을 직접 추가해야 합니다.
- 이 `static()` 설정은 개발 단계(DEBUG=True)에서만 동작합니다. 실제 서비스에서는 웹 서버나 S3 같은 저장소가 파일을 제공합니다. (=> 참고: AWS 인프라 이해하기 챕터)

이미지 업로드 구현 (1/8)

- Article 모델 클래스에 image 필드 작성
- `blank=True` 속성을 작성해 폼에서 이미지를 비워도 유효성 검사를 통과하도록 설정
  - 게시글 작성 시 이미지 첨부 없이도 작성 할 수 있도록 하기 위함

blank 속성
폼 유효성 검사 시 해당 필드를 필수 또는 선택 사항으로 만들지 결정하는 속성

<개념1_Django_Static files_blank 속성>

```python
# articles/models.py

class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
    image = models.ImageField(upload_to='images/', blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

※ 기존 필드 사이에 작성해도 실제 테이블 생성 시에는 가장 우측(뒤)에 추가됩니다.

이미지 업로드 구현 (2/8)

- migration 진행 과정 중 에러 발생

Pillow
파이썬에서 이미지를 열고, 크기 조절이나 필터 적용 같은 다양한 편집 작업을 수행한 후 저장할 수 있게 해주는 필수 이미지 처리 라이브러리

<개념2_Django_Static files_Pillow 라이브러리>

```
$ python manage.py makemigrations

SystemCheckError: System check identified some issues:

ERRORS:
articles.Article.image: (fields.E210) Cannot use ImageField because Pillow is not installed.
        HINT: Get Pillow at https://pypi.org/project/Pillow/ or run command "python -m pip
install Pillow".
```

※ ImageField를 사용하려면 반드시 Pillow 라이브러리가 필요합니다.

이미지 업로드 구현 (3/8)

- Pillow 설치 후 migrations 재진행

```
$ pip install pillow
$ python manage.py makemigrations
$ python manage.py migrate
$ pip freeze > requirements.txt
```

이미지 업로드 구현 (4/8)

- form 요소의 enctype 속성 추가

enctype 속성
form 데이터가 서버로 제출될 때,
해당 데이터가 어떤 형식으로
인코딩 될지 지정하는 속성

<개념3_Django_Static files_enctype 속성>

```
<!-- articles/create.html -->

<h1>CREATE</h1>
<form action="{% url 'articles:create' %}" method="POST" enctype="multipart/form-data">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

※ 파일 업로드 시에는 반드시 multipart/form-data로 설정해야 합니다.

https://developer.mozilla.org/en-US/docs/Web/HTML/Element/form

이미지 업로드 구현 (5/8)

- ModelForm의 2번째 인자로 요청 받은 파일 데이터 작성

```
# articles/views.py

def create(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST, request.FILES)
        …
```

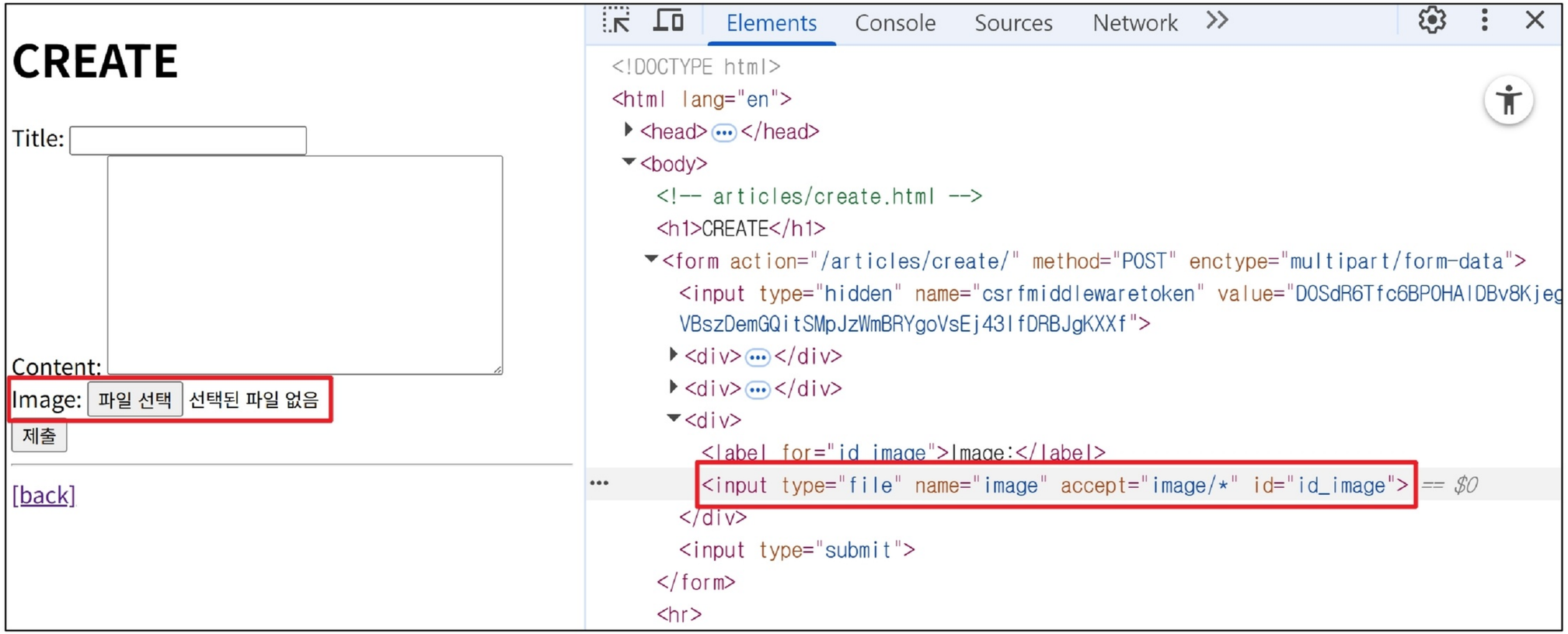
TIP

ModelForm의 두번째 인자로 파일을 작성한 이유는?

- ModelForm의 상위 클래스인 BaseModelForm의 생성자 함수의 2번째 위치 인자로 파일을 받도록 설정 되어 있습니다.

이미지 업로드 구현 (6/8)

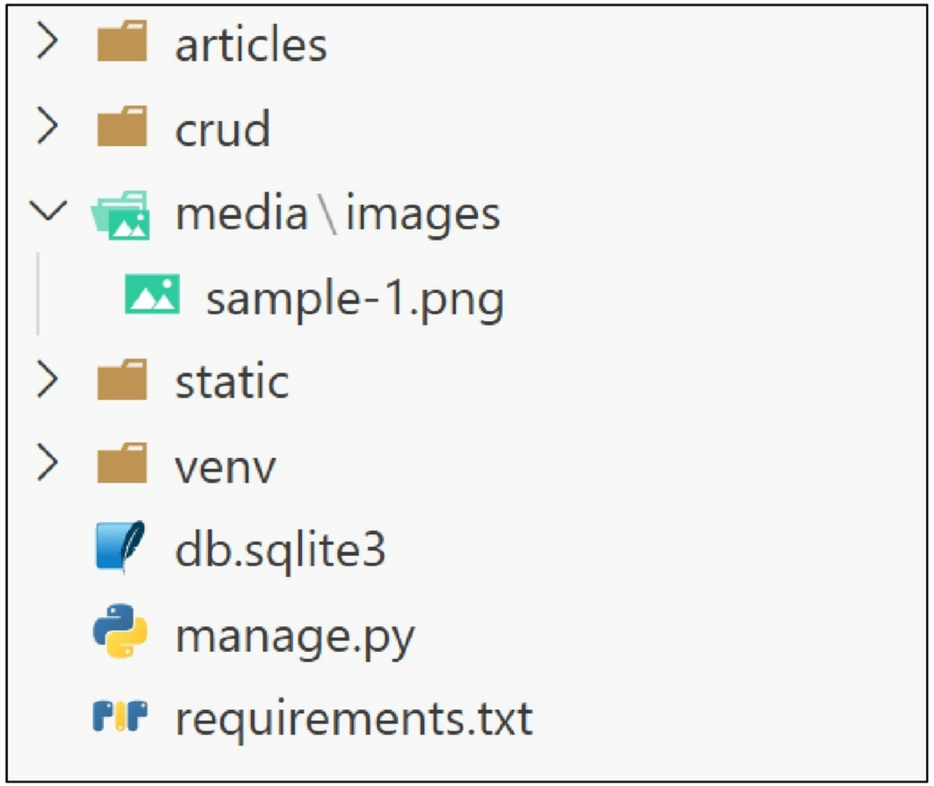
- 게시글 작성 페이지에서 새로 추가된 input 태그 확인



<그림8_Django_Static files_파일 업로드 input 출력>

이미지 업로드 구현 (7/8)

- 이미지 업로드 후 이미지 파일 위치 확인
  - `MEDIA_ROOT` 경로에 업로드 된 이미지 파일

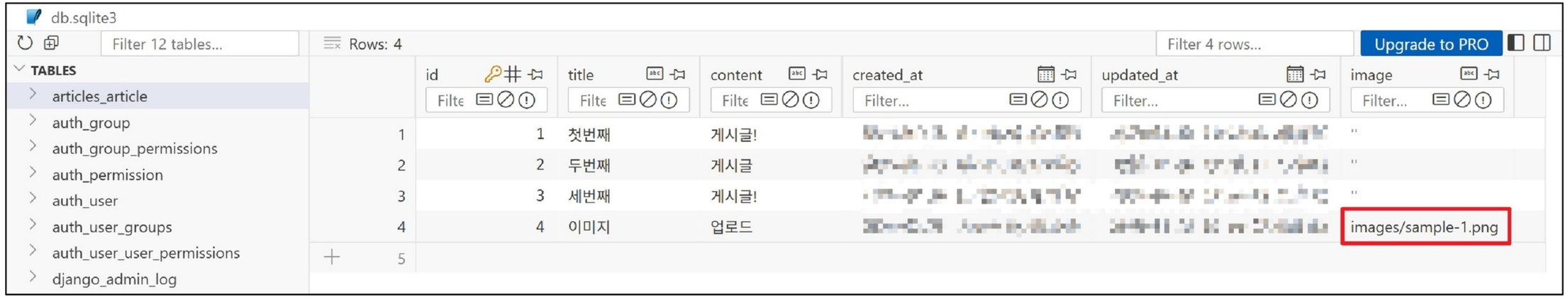


<그림9_Django_Static files_MEDIA_ROOT 결과>

이미지 업로드 구현 (8/8)

- 이미지 업로드 후 DB 확인
  - 이미지 파일 자체가 데이터베이스에 저장되는 것이 아닌 `upload_to` 경로를 기준으로 한 이미지 파일의 경로(문자열)만 저장

<그림10_Django_Static files_DB 문자열 저장 결과>



TIP

- **이 방식은 데이터베이스의 용량을 가볍게 유지하고 성능을 높이는 데 매우 효율적입니다.**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

업로드 이미지 제공

게시글 상세 페이지에 업로드 한 이미지 제공하기 (1/2)

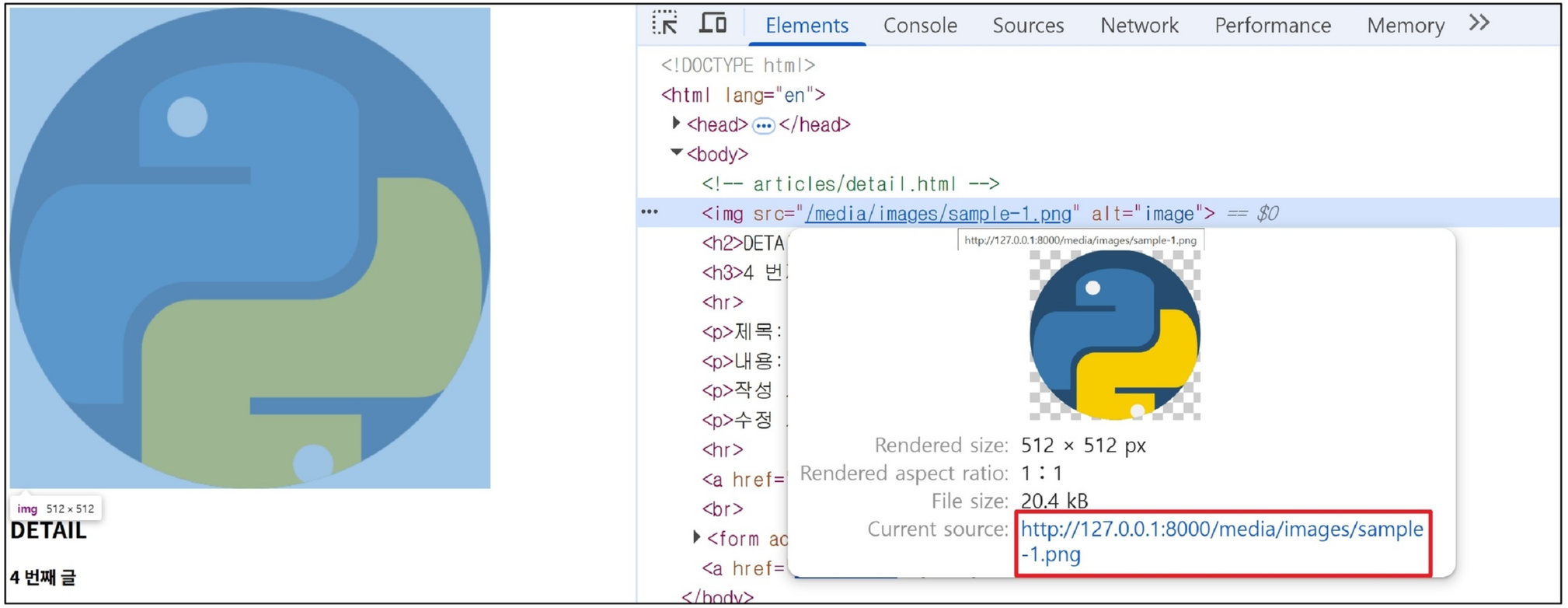
- ImageField의 .url 속성
  - 업로드 파일의 웹 주소
  - ImageField나 FileField에 저장된 파일 객체에서 .url 속성을 사용하면, 해당 파일을 웹에서 접근할 수 있는 전체 URL 주소를 얻을 수 있음

```
<!-- articles/detail.html -->

{% if article.image %}
  <img src="{{ article.image.url }}" alt="image">
{% endif %}
```

※ 이미지를 업로드하지 않은 게시물에서 `article.image.url`을 사용하면
연결된 파일이 없다는 에러(ValueError)가 발생합니다.
이미지 데이터가 있는 경우만 이미지를 출력할 수 있도록 if 태그를 활용합니다.

게시글 상세 페이지에 업로드 한 이미지 제공하기 (2/2)

- 업로드 이미지 출력 확인 및 URL 확인
  - http://127.0.0.1:8000/media/images/sample-1.png



<그림11_Django_Static files_업로드 이미지 출력 결과>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

업로드 이미지 수정

업로드 이미지 수정 (1/2)

- 수정 페이지 form 요소에 enctype 속성 추가

```
<!-- articles/update.html -->

<h1>UPDATE</h1>
<form action="{% url 'articles:update' article.pk %}" method="POST" enctype="multipart/form-data">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

업로드 이미지 수정 (2/2)

- update view 함수에서 업로드 파일에 대한 추가 코드 작성

```
# articles/views.py

def update(request, pk):
    article = Article.objects.get(pk=pk)
    if request.method == 'POST':
        form = ArticleForm(request.POST, request.FILES, instance=article)
…
```

참고

upload_to 활용

upload_to 속성 심화 활용

- 단순한 문자열 경로 외에도, 업로드 경로를 동적으로 생성하는 두 가지 유용한 방법을 제공

1. 날짜를 이용한 경로 구성
2. 함수를 이용한 동적 경로 생성

1. 날짜를 이용한 경로 구성

- strftime()의 형식 코드를 사용하여 파일이 업로드된 날짜를 기준으로 폴더를 자동으로 생성할 수 있음
  - **%Y**: 4자리 연도
  - **%m**: 2자리 월
  - **%d**: 2자리 일

```
class Photo(models.Model):
    # 2100년 1월 1일에 업로드하면 '2100/01/01/' 폴더에 저장됨
    image = models.ImageField(blank=True, upload_to='%Y/%m/%d/')
```

2. 함수를 이용한 동적 경로 생성

- 더 복잡한 로직으로 경로를 만들고 싶을 때는, **upload_to**에 함수를 직접 전달할 수 있음
- 이 함수는 두 가지 인자를 받음
  - **instance**: 파일이 첨부된 모델의 인스턴스 (해당 게시글 객체 등)
  - **filename**: 업로드 된 파일의 원본 이름

```python
# 경로 생성 함수 정의
def articles_image_path(instance, filename):
    # instance.user.username을 통해 게시글 작성자의 이름을 가져옴
    # 예: 'images/ssafy_user/my_photo.jpg' 와 같은 경로를 반환
    return f'images/{instance.user.username}/{filename}'

class Article(models.Model):
    user = ...
    image = models.ImageField(blank=True, upload_to=articles_image_path)
```

※ user 필드는 이후 Authentication 수업에서 학습합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

AWS 인프라 이해하기

AWS (Amazon Web Services)

- 아마존이 제공하는 클라우드 컴퓨팅 플랫폼
- 서버, 스토리지, 데이터베이스 같은 IT 인프라를 직접 구매하지 않고, 인터넷을 통해 필요한 만큼 빌려 쓰는 서비스

AWS 핵심 서비스 3가지

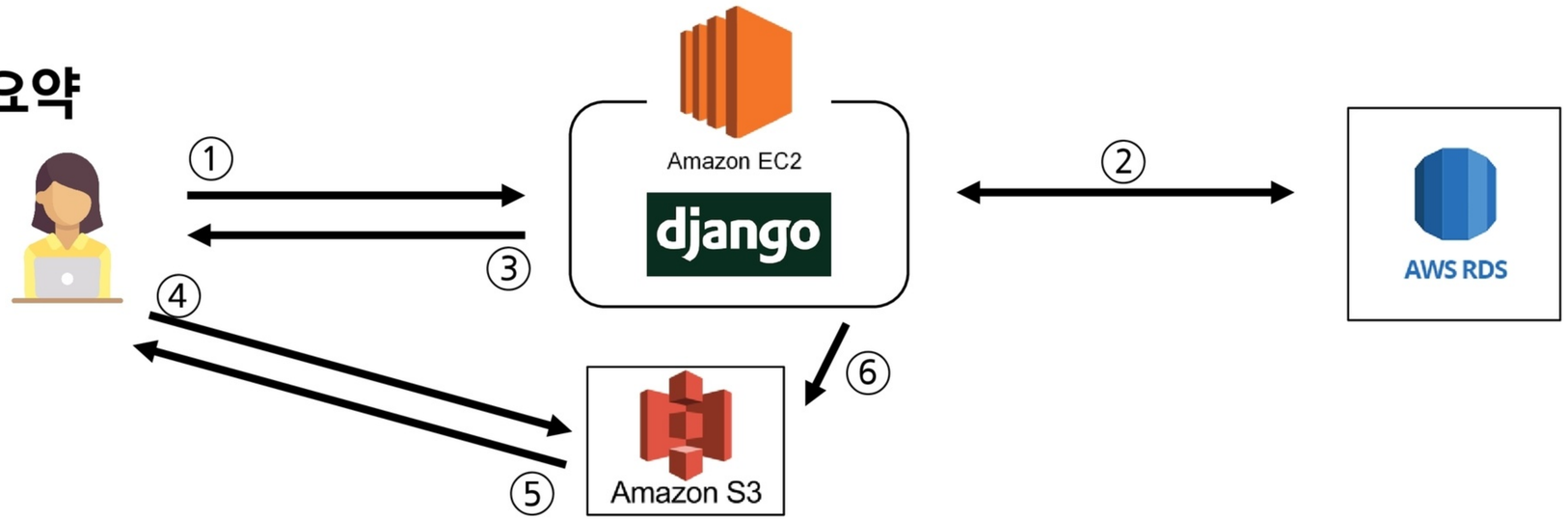
서비스	역할	설명
EC2	가상 서버	클라우드에 생성하는 고성능 컴퓨터
S3	파일 저장소	이미지, 동영상 등 모든 파일을 보관하는 객체 스토리지
RDS	데이터베이스	데이터를 체계적으로 관리하는 관계형 데이터베이스

<표2_Django_Static files_AWS 서비스>

EC2, S3, RDS를 활용한 웹 서비스 구축

- EC2 (서버)
  - 웹 애플리케이션(Django)이 실행되는 컴퓨터의 역할
- S3 (파일 저장소)
  - 사용자가 올린 이미지(미디어 파일)나 사이트의 로고·CSS(정적 파일)를 보관하는 외부 창고 역할
- RDS (데이터베이스)
  - 회원 정보, 게시글 등 중요한 데이터를 기록하고 관리하는 장부 역할

데이터 흐름도 요약



<그림12_Django_Static files_클라우드 플랫폼 데이터 흐름도>

1. **사용자 ➡️ EC2 (HTTP Request)**
   사용자가 브라우저를 통해 웹사이트에 접속(요청)합니다. 이 요청은 가장 먼저 EC2에 도달
2. **EC2 ↔ RDS (DB Query)**
   EC2의 Django 애플리케이션은 요청을 처리하기 위해 필요한 데이터(예: 게시글, 사용자 정보)를 RDS에 조회하거나 새로운 데이터를 저장
3. **EC2 ➡️ 사용자 (HTML Response)**
   EC2는 데이터베이스에서 가져온 정보와 HTML 템플릿을 조합하여 완전한 웹 페이지(HTML)를 만들어 사용자에게 응답
   이 HTML 페이지 안에는 이미지, CSS, JS 파일의 주소가 S3를 가리키도록(`src="https://...s3..."`) 포함되어 있음
4. **사용자 ➡️ S3 (File Request)**
   HTML을 받은 사용자의 브라우저는 페이지를 완전하게 그리기 위해, HTML에 명시된 S3 주소로 직접 이미지나 CSS 파일을 요청/ EC2를 거치지 않음
5. **사용자 ⬅️ S3 (File Response)**
   S3는 요청 받은 파일을 사용자 브라우저에 직접 전송
6. **(파일 업로드 시) 사용자 ➡️ EC2 ➡️ S3**
   사용자가 이미지를 업로드하면, 요청은 EC2로 감. EC2의 Django 앱은 파일을 받아 S3에 저장

클라우드 컴퓨팅 플랫폼을 사용하는 이유

- 비용 절감
  - 물리 장비를 직접 구매할 필요 없이, 사용한 만큼만 비용을 지불
- 유연성과 확장성
  - 필요에 따라 서버 사양이나 저장 공간 용량을 몇 번의 클릭만으로 자유롭게 조절할 수 있음
- 글로벌 서비스
  - 전 세계에 위치한 데이터 센터를 통해 어떤 국가의 사용자에게도 빠르고 안정적인 서비스를 제공할 수 있음

BaseModelForm

BaseModelForm의 생성자 함수 인자 구성

- ModelForm의 상위 클래스 BaseModelForm의 생성자 함수 키워드 인자 순서 참고하기

```
class BaseModelForm(BaseForm):
    def __init__(
        self,
        data=None,
        files=None,
        auto_id="id_%s",
        prefix=None,
        initial=None,
        error_class=ErrorList,
        label_suffix=None,
        empty_permitted=False,
        instance=None,
        use_required_attribute=None,
        renderer=None,
    ):
```

<그림13_Django_Static files_BaseModelForm 생성자 함수 구성>

실습

Static files

- 3238. SNS 서비스 만들기 - STATIC
- 3039. static 이미지 제공과 코드 개선
- 2694. 장소 Media & Static 파일 기본 설정 예제

Media files

- 3038. 이미지 업로드 기능 구현
- 2695. 업로드 된 Media 파일 URL 설정 예제

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 정적 파일에 대한 가장 올바른 설명은?

a) 사용자가 업로드하는 파일
b) 서버 측에서 변경 없이 고정적으로 제공되는 파일
c) 데이터베이스에 저장되는 정보
d) 사용자에 따라 내용이 바뀌는 파일

2 Django settings.py에서 STATIC_URL의 역할은 무엇인가?

a) 정적 파일이 저장된 실제 폴더 경로
b) 웹에서 정적 파일에 접근하기 위한 URL 접두사
c) 데이터베이스 연결 주소
d) 미디어 파일 업로드 경로

3 ImageField를 사용한 모델에서 데이터베이스에 실제로 저장되는 것은 무엇인가?

a) 이미지 파일의 바이너리 데이터
b) 이미지 파일의 경로 문자열
c) Pillow 라이브러리 객체
d) 이미지 파일의 base64 인코딩 값

4 ImageField를 정의할 때, MEDIA_ROOT 안에서 파일이 저장될 세부 경로를 지정하는 인자는?

a) `path`
b) `default`
c) `upload_to`
d) `media_path`

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 article이라는 변수에 ImageField를 가진 객체가 전달되었을 때, 이 이미지를 HTML에 표시하기 위한 <img> 태그의 src 속성을 Django 템플릿 태그를 사용하여 완성하시오. (단, ImageField의 인스턴스는 image이다.)

```
<img src=" ___(?)___ " alt="image">
```

6 아래 코드에서 ①번과 ②번에 각각 어떤 settings.py 변수가 사용되어야 하는지 작성하시오.

```
urlpatterns = [
    …
] + static(①, document_root= ②)
```

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

7 Django 템플릿에서 {% static %} 태그를 사용하기 위해 가장 먼저 선언해야 하는 태그는?

a) `{% import static %}`
b) `{% prepare static %}`
c) `{% use static %}`
d) `{% load static %}`

8 Django가 정적 파일을 찾을 때, 각 앱의 static 폴더 외에 추가적인 탐색 경로를 지정하는 설정은?

a) `STATIC_PATH`
b) `STATIC_ROOT`
c) `STATICFILES_DIRS`
d) `EXTRA_STATIC_FILES`

9 서버에 파일이 존재하더라도, 웹 브라우저가 그 파일을 화면에 표시하기 위해 반드시 필요한 것은?

a) 파일의 압축
b) 데이터베이스 레코드
c) 해당 파일에 접근 가능한 고유 URL
d) 파일에 대한 관리자 승인

10 Django의 ImageField를 사용하기 위해 반드시 설치해야 하는 파이썬 이미지 처리 라이브러리의 이름은 무엇인지 작성하시오.

확인 문제

정답 및 해설

1. b) 서버 측에서 변경 없이 고정적으로 제공되는 파일 2. b) 웹에서 정적 파일에 접근하기 위한 URL 접두사
3. b) 이미지 파일의 경로 문자열 4. c) `upload_to` 5. `{{ article.image.url }}`
6. ① `settings.MEDIA_URL`, ② `settings.MEDIA_ROOT` 7. d) `{% load static %}` 8. c) `STATICFILES_DIRS`

1. 정적 파일은 사이트 개발자가 미리 준비해 둔 파일로, 서버가 실행되는 동안 내용이 변하지 않고 고정적으로 제공됩니다. (예: CSS, 로고 이미지)
2. STATIC_URL은 실제 파일 시스템 경로가 아닌, 웹 브라우저가 정적 파일에 접근할 때 사용하는 URL의 시작 부분을 정의합니다.
3. Django의 ImageField는 이미지 파일 자체를 DB에 저장하지 않습니다. DB에는 해당 파일이 서버 어디에 저장되어 있는지를 나타내는 경로 정보(문자열)만 저장하여 효율성을 높입니다.
4. upload_to 인자는 ImageField가 MEDIA_ROOT 내에서 파일을 저장할 세부 경로를 지정하는 항목입니다.
5. 모델의 이미지 필드 객체에서 .url 속성을 사용하면 해당 이미지에 접근할 수 있는 전체 URL을 얻을 수 있습니다.
6. static() 함수는 MEDIA_URL(웹 주소)로 들어온 요청을 MEDIA_ROOT(실제 폴더)에 연결해줍니다.
7. {% static %} 태그를 사용하려면, 템플릿 최상단에 {% load static %}을 선언하여 Django에게 static 관련 태그 라이브러리를 사용하겠다고 알려줘야 합니다.
8. STATICFILES_DIRS는 각 앱의 static/ 디렉토리 외에, Django가 정적 파일을 추가로 찾아볼 경로 목록을 정의하는 설정입니다.

확인 문제

정답 및 해설

9. c) 해당 파일에 접근 가능한 고유 URL
10. Pillow

9. 웹의 동작 원리상, 서버 컴퓨터에 파일이 있더라도 브라우저가 그 파일에 접근할 수 있는 고유한 주소(URL)가 없다면 파일을 요청하고 받아올 수 없습니다.

10. ImageFieId는 이미지 파일의 유효성 검사 및 정보 처리를 위해 Pillow 라이브러리를 필요로 합니다.

활동 정리

핵심 키워드

개념	설명	예시
정적 파일 (Static Files)	웹 사이트 개발 시 미리 준비된 파일. (예: CSS, JS, 로고 이미지)	`STATIC_URL, STATICFILES_DIRS`
미디어 파일 (Media Files)	사용자가 웹 사이트를 통해 업로드하는 파일. (예: 프로필 사진, 게시글 첨부 파일)	`MEDIA_URL, MEDIA_ROOT`
`STATICFILES_DIRS`	기본 경로(app/static/) 외에 추가적인 정적 파일 경로를 지정하는 설정.	`STATICFILES_DIRS = [BASE_DIR / 'static']`
`MEDIA_ROOT`	업로드된 미디어 파일이 저장될 서버의 실제 (절대) 경로.	`MEDIA_ROOT = BASE_DIR / 'media'`
`MEDIA_URL`	MEDIA_ROOT에 저장된 파일을 웹 페이지에서 접근할 때 사용할 URL.	`MEDIA_URL = 'media/'`
`ImageField`	이미지 파일 업로드를 위한 모델 필드.	`image = models.ImageField(upload_to='images/')`
`enctype`	파일이나 이미지를 서버로 전송할 때 사용해야 하는 <form> 태그 속성	`<form enctype="multipart/form-data">`

<표3_Django_Static files_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- 정적 파일(Static Files): "개발자가 미리 준비한 파일"
  - CSS, JavaScript, 로고, 아이콘처럼 사이트의 디자인과 기능을 위해 개발자가 미리 준비해두는 파일입니다.
- 미디어 파일(Media Files): "사용자가 올리는 파일"
  - 게시글의 첨부파일, 사용자의 프로필 사진처럼 사이트 운영 중에 사용자가 직접 업로드하는 파일입니다.
  - settings.py에 MEDIA_URL, MEDIA_ROOT를 설정하고, urls.py에 URL 패턴을 추가해야 합니다.
  - 모델에서는 ImageField 또는 FileField를 사용하고, form 태그에는 enctype="multipart/form-data" 속성을 반드시 추가해야 합니다.

활동 정리

밋밋했던 우리 웹사이트, 이제는 생기가 넘칩니다!

- 정적 파일과 미디어 파일 관리법을 익힘으로써 우리는 훨씬 더 동적이고 완성도 높은 웹 사이트를 만들 수 있게 되었습니다.

1. 로고 이미지 넣기

   프로젝트 최상위 static 폴더를 만들고 `STATICFILES_DIRS`에 등록한 뒤, `{% static 'logo.png' %}` 태그로 간단히 로고를 불러올 수 있습니다.

2. 게시글 이미지 업로드

   `models.py`에 `ImageField`를 추가하고, `MEDIA_ROOT`와 `MEDIA_URL` 설정을 통해 사용자가 업로드한 이미지를 성공적으로 화면에 보여줄 수 있습니다.

3. CSS로 꾸미기

   `style.css` 파일을 `static` 폴더에 넣고 `<link>`태그로 연결하여 웹 페이지 전체의 디자인을 구성할 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다